

Multi-Class Infrastructure Damage Classification using Aerial Remote Sensing

Chennakeshava Akhil Pillalamarri
MSDS-696 Data Science Practicum II
Regis University, Denver, CO, USA
cpillalamarri@regis.edu

Abstract

Post-Disaster structural damage analysis is an important actor of emergency response operations. Traditional manual assessment methods performed by Civil Air Patrol volunteers and FEMA staff are slow, expensive, and difficult to scale on the large disaster-affected areas. This project presents a deep learning pipeline for automating multi-class damage classification of low-altitude aerial disaster imagery using Convolutional Neural Networks. The LADI v2 federal government dataset [1] consisting of 9,969 aerial images collected across federally declared US disasters from 2015 to 2023, is used to classify images into three categories used in the official FEMA Preliminary Damage Assessment scale [2]: Intact, Major Damage, and Structural Failure. Five architectures inspired by VGG-16 [3], ResNet-18 [4], InceptionV1 [5], and DenseNet-40 [6] were benchmarked alongside a retrained custom baseline. Severe class imbalance was addressed using seven strategies including Focal Loss [7], a Balanced Batch Generator [8], targeted data augmentation [9], automatic threshold adjustment [10], and Macro F1 monitoring. The DenseNet-40 Inspired DamageCNN achieved the best overall validation Macro F1 of 0.8143, which is a 133% improvement from the baseline of 0.33, while correctly identifying 84.62% of Structural Failure test images. The Custom DamageCNN Retrained on augmented data achieved the best Major Damage detection at 45.45%, showing that targeted augmentation was equally important as deep architectures. GRAD-CAM [11] visualizations gave pixel-level detail for government deployment contexts.

I. INTRODUCTION / BACKGROUND

Structural damage analysis post-disaster is an important step needed in any emergency response operation. Right after disasters like hurricanes, earthquakes, floods, or wildfires, emergency response teams like FEMA must assess the damage intensity as quickly as possible among the large areas. currently this process is time-consuming, resource-intensive and difficult to scale [2].

Recent improvements in deep learning and compute vision techniques have shown the potential of CNN architectures usage for automated tasks in image classification. CNNs learn directly from raw image pixels, recognizing more complex visual patterns at each layer making them effective for identifying damage in aerial disaster photos. CNN architectures like VGG, ResNet, Inception, and DenseNet have better improved approaches towards feature learning with deep stacked convolutions, residual skip connections, multi-scale parallel processing, and reusing dense features.

This project uses five CNN architectures to classify aerial images into three distinct classes which are Intact, Major Damage and Structural Failure. All the models are trained on LADI (Low-Altitude Disaster Images) dataset [1], federal government dataset collected by Civil Air Patrol across federally declared US disasters, and developed by the MIT Lincoln Laboratory. The damage classes are used in the official FEMA preliminary damage assessment scale [2].

Main challenge in this project is that there is a severe class imbalance where 81.5% of images are in the intact damage class, whereas major damage has only about 5.9% and structural failure only 12.6%. Seven strategies were applied to address the class imbalance issue which includes Focal Loss, Balanced Batch Sampling, Targeted Data Augmentation, and Threshold Adjustment. GRAD-CAM visualizations were generated to provide pixel-level explanations for the model predictions on the images using heatmap overlays on it.

II. PROBLEM STATEMENT

After any natural disaster, emergency response teams need to find out which buildings and infrastructure are damaged as soon as possible so they can send help to the right places. Currently, this is done manually by Civil Air Patrol volunteers and FEMA staff who assess thousands of aerial images one by one. This process takes time, is expensive, and is very difficult to scale among large disaster-affected areas [2].

The goal of this project is to build an automated system that looks at these aerial images and classifies those into any of the three damage categories which are Intact, Major Damage, or Structural Failure using deep learning. However, building this system comes with some challenges that needed to be resolved.

The first challenge is class imbalance. In the LADI v2 dataset [1], 81.5% of images show intact undamaged class, while only 5.9% show Major Damage and 12.6% Structural Failure. This shows the reality of real disasters where most buildings survive. However, it creates a serious problem for training a model, a model that simply predicts every image as Intact would score 81.5% accuracy without detecting a single damaged building. This makes overall accuracy a completely misleading evaluation metric for this process.

Moreover, second challenge is choosing the right evaluation metric. Because of the class imbalance, Macro F1 score was chosen as the primary evaluation metric. Macro F1 calculates the F1 score separately for each class and averages them equally, so the model only receives a good score if it performs well across all three damage categories, not just the majority Intact class.

The third challenge is the limited quantity of images from the minority class in the test set. The test set includes just 22 Major Damage and 26 Structural Failure images, while it has 1,001 Intact images. Due to the low numbers, a single incorrect prediction can shift the accuracy by 4 to 5 percentage points. This is not an error in the data collection method, it merely indicates the reality that most structures endure disasters. Due to this constraint, the validation set comprising 3,210 images serves as the primary metric for assessing model performance in this project, offering a significantly more dependable and consistent evaluation.

The fourth challenge is model explainability. For a system to be useful in real government deployment contexts such as FEMA damage triage, the model's decisions must be explainable and visually justified. GRAD-CAM [11] visualizations were generated to show which specific regions of each aerial image most influenced the model's damage classification prediction.

The fifth challenge involves training every architecture entirely from the beginning without using any pretrained weights. This indicates that each model needed to acquire all damage-associated visual characteristics solely from the 9,969 LADI v2 images, necessitating meticulous design decisions to avoid overfitting on a comparatively small and unbalanced dataset.

III. RELATED WORK

Many published research works are used as a foundation for the techniques and methods used in this project. Each work is directly linked to a specific part of the pipeline.

A. Dataset

The dataset applied in this project, LADI v2, was presented by Scheele et al. [1]. It features low-altitude aerial disaster photographs gathered by Civil Air Patrol pilots during actual US disasters, and created by MIT Lincoln Laboratory. The images are categorized according to the official FEMA damage assessment protocol [2], which specifies the three damage classes applied in this project: Intact, Major Damage, and Structural Failure

B. Image Preprocessing

All preprocessing steps applied in this project are sourced from a published research paper. Yusuf et al. [12] demonstrated that using Gaussian blur and histogram equalization before training a CNN on aerial

images helps in removing noise and improving image contrast, resulting in improved model performance. Jintasuttisak et al. [13] showed that transforming aerial images from RGB to HSV color space improves the distinction of visual features, explaining the model's detection of structural patterns. Abdelmonem et al. [14] showed that resizing and normalizing image is a typical and efficient preprocessing step for classification tasks in remote sensing.

C. CNN Architectures

The four architectures implemented in this project are inspired from published studies. Simonyan and Zisserman [3] showed that combining multiple small 3×3 convolution layers gets better performance compared to using a smaller number of larger convolutions, this concept is fundamental to the VGG-16 inspired architecture. He et al. [4] introduced skip connections in ResNet, allowing the learning signal to next layers and directly access earlier sections of the network, addressing the issue of deep networks stopping to learn. Szegedy et al. [5] proposed using different convolution sizes at the same time in one layer in InceptionV1, allowing the model to examine the image across various scales simultaneously. Huang et al. [6] introduced DenseNet, where every layer is directly connected to every other layer, ensuring that no learned feature is discarded, which is especially beneficial for smaller datasets.

D. Class Imbalance Handling

Four published studies support the strategies used to address class imbalance in this project. Lin et al. [7] proposed Focal Loss, which allows the model to focus less on easy accurate predictions and more on the difficult minority class images it frequently misclassifies. Buda et al. [8] examined class imbalance in CNNs and showed that ensuring equal class representation in each training batch greatly improve minority class detection, this principle explains the Balanced Batch Generator developed in this project. Shorten and Khoshgoftaar [9] examined image augmentation methods for deep learning and demonstrated that producing additional training images for minority classes via rotation, flipping, brightness modification, and cropping can greatly improve model efficiency on imbalanced datasets. Leevy et al. [10] explored techniques for solving class imbalance issue, including adjusting the decision threshold in inference, which was used in this project to improve minority class detection without requiring retraining

E. Explainability

Selvaraju et al. [11] presented GRAD-CAM Visualizations, which produce a heat map that overlays an image to precisely indicate the regions of the image the model focused on while making its prediction. Red and yellow regions indicate where the model focused greatly. This was used in this project to make the model's decisions visually explainable, which is important for any system intended for real government use.

IV. METHODOLOGY

A. Dataset Loading and Label Mapping

The LADI v2 dataset [1] was loaded directly from HuggingFace using the datasets library MITLL/LADI-v2-dataset. The dataset provides 12 binary infrastructure label columns for each image. A total of three damage categories were identified from these columns based on the following criteria: if `buildings_affected_or_greater` is True, the image is classified as Structural Failure; if `buildings_minor_or_greater` is True, the image is classified as Major Damage; and all other images are classified as Intact. The processed images were stored and structured into train, validation, and test directories, each containing a subfolder for each category. The split for training, validation, and testing was established by MIT Lincoln Laboratory using an 80/9/11 ratio. The test set remained entirely unaltered during the whole project to guarantee an impartial final assessment

TABLE I
DATASET SPLIT AND CLASS DISTRIBUTION BEFORE AUGMENTATION

Split	Intact	Major Damage	Structural Failure	Total
Train	6,542	476	1,012	8,030
Validation	710	65	115	890
Test	1,001	22	26	1,049

B. Image Preprocessing Pipeline

Prior to training any model, every image underwent a four-step preprocessing pipeline. Every step was selected according to a research paper that applied the same method to aerial or remote sensing imagery

- 1) **Gaussian Blur (5×5):** It Removes sensory noise which is common in low-altitude aerial photography while keeping important structural boundary information intact [12].
- 2) **Histogram Equalization (YUV channel):** Improving contrast in aerial images which are overexposed [12].
- 3) **HSV Color Transformation:** Converting the images from RGB to HSV color space to separates structural damage color signatures, more clearly than RGB [13].
- 4) **Resize and Normalize:** All the images are resized and pixel values are normalized to the range of 0 to 1 to standardize the inputs across all the models [14]. The base CNN was initially trained at 224×224 pixels. All benchmarking models were trained at 160×160 pixels

C. Class Imbalance Strategies

The baseline model trained on the dataset predicted almost everything as Intact, achieving 100% accuracy while completely failing to detect damaged buildings. Seven strategies were applied to address this class-imbalance problem.

- 1) **Focal Loss ($\gamma=2.0$ and $\gamma=3.0$):** Automatically makes the model pay less attention to easy correct predictions and more focus towards hard minority classes [7]. Gamma=2.0 was used for the base CNN and gamma=3.0 was used for all benchmarking models.
- 2) **Balanced Batch Generator:** A custom generator was built to force use exactly 11 images from each class into every training batch, 33 images total, making sure the model always trains on equal numbers of all three classes regardless of their size in the dataset [8].
- 3) **Macro F1 Monitoring:** Early stopping and learning rate reduction monitor validation Macro F1 rather than overall accuracy, to make sure the model is only rewarded for improving performance across all three classes equally.
- 4) **Threshold Adjustment:** An automatic threshold search was performed across values between 0.05 and 0.25 to find the confidence threshold that maximizes minority class detection at inference time [10].
- 5) **LR Reduction on Plateau:** ReduceLROnPlateau with factor=0.5 and patience=4 was applied on validation Macro F1, automatically cutting the learning rate in half when performance stops improving to allow better weight updates.
- 6) **Targeted Augmentation:** Major Damage training images were augmented from 476 to 9,044 images (19×) and Structural Failure from 1,012 to 11,132 images (11×) using rotation up to 40 degrees, horizontal and vertical flipping, brightness adjustment between 0.7 and 1.3, random zoom between 0.8 and 1.0, and random cropping [9]. Augmentation was applied using PIL and NumPy to preserve full control over the augmented image count.
- 7) **Validation Augmentation:** Minority classes were also augmented in the validation set to provide more balanced and reliable performance monitoring during training [9].

TABLE II
DATASET COUNTS AFTER AUGMENTATION

Split	Intact	Major Damage	Structural Failure	Total
Train	6,542	9,044	11,132	26,718
Validation	710	1,235	1,265	3,210
Test	1,001	22	26	1,049

D. CNN Architectures

A total of five CNN architectures were used without any pretrained weights. Each model was built using TensorFlow 2.21. All benchmark models followed similar training configurations with an input dimension of $160 \times 160 \times 3$, Focal Loss with parameters $\gamma=3.0$ and $\alpha=0.25$ [7], the Adam optimizer set with a learning rate of $1e-4$, a balanced batch generator delivering 33 images per batch, early stopping with a patience of 8 monitoring the validation Macro F1, and ReduceLROnPlateau with a patience of 4 and a reduction factor of 0.5.

1) *Custom DamageCNN*: The Custom DamageCNN functions as the baseline architecture, consisting of four sequential convolutional blocks featuring filters of 32, 64, 128, and 256. Each block contains a Conv2D layer for extracting features, Batch Normalization to ensure training stability, MaxPooling 2×2 to progressively reduce spatial dimensions, and Dropout rates of 0.25, 0.30, 0.40, and 0.50 respectively to reduce overfitting. The model has a total of 457,923 parameters and was first trained with an input size of $224 \times 224 \times 3$, utilizing a Focal Loss γ of 2.0. It was later retrained at $160 \times 160 \times 3$ with $\gamma=3.0$ using the fully augmented dataset

2) *VGG-16*: Inspired by Simonyan and Zisserman [3], this model extends the base architecture with double and triple stacked Conv2D layers in Blocks 3 and 4, giving 12 total convolutional layers. This replicates the VGG-16 design principle that deep stacked 3×3 convolutions outperform shallower networks with larger convolution kernels. This model was trained on the original unaugmented dataset at $160 \times 160 \times 3$ with Focal Loss $\gamma=2.0$ to establish a depth-only benchmark before augmentation was applied.

3) *ResNet-18*: Motivated by He et al. [4], this model incorporates residual skip connections across three sets of blocks with 64, 128, and 256 filters. Every group consists of two residual blocks, with the input of each block being directly added back to its output via an identity shortcut connection. When spatial dimensions differ across groups, a 1×1 convolution modifies the shortcut to align the dimensions. Skip connections address the vanishing gradient issue by creating direct gradient routes to earlier layers, allowing for efficient training of deeper networks.

4) *InceptionV1*: Motivated by Szegedy et al. [5], this architecture utilizes parallel convolutions at three different scales concurrently in each inception module, 1×1 , 3×3 , and 5×5 convolutions in addition to a MaxPooling path. The results from all four branches are joined together along the channel axis. This multi-scale method simultaneously captures damage characteristics at various spatial resolutions, which is advantageous for aerial imagery, as the same damage type may manifest at significantly different scales based on flight altitude

5) *DenseNet-40*: Inspired by Huang et al. [6], this model uses four dense blocks with 4, 6, 6, and 4 layers respectively at a growth rate of 16, connected by transition blocks consisting of Batch Normalization, 1×1 Conv, and AveragePooling 2×2 with reduction factor 0.5. Every layer within each dense block receives concatenated feature maps from all preceding layers, meaning no learned feature is ever lost throughout the network. This architecture has only 350,303 total parameters, the lightest model in the benchmark, while achieving the best overall performance.

E. GRAD-CAM Explainability

GRAD-CAM [11] visualizations were generated for the Custom DamageCNN Retrained model to provide pixel-level explanations for damage classification decisions. Due to incompatibility between

Keras 3.x Sequential models and the standard Functional API gradient model approach, a layer-by-layer execution strategy was implemented using `tf.GradientTape` to directly watch the output of the last convolutional layer. The gradient of the top predicted class with respect to the convolutional output was computed, globally pooled, and used to weight the feature maps. The resulting heatmap was ReLU-activated, normalized, resized to 160×160, and overlaid on the original image using OpenCV’s COLORMAP_JET colormap with alpha=0.4 blending. An annotated MP4 video was generated at 2 frames per second for 45 test images showing the original image, GRAD-CAM overlay, true class, predicted class, and confidence score.

V. DESCRIPTION OF DATA

A. Overview of the Dataset

The dataset used in this project is LADI v2 (Low Altitude Disaster Imagery version 2), presented by Scheele et al. [1]. This is the only federal aerial disaster imagery dataset available publicly in the United States, gathered by Civil Air Patrol pilots surveying actual US disaster areas from 2015 to 2023. The dataset was created by MIT Lincoln Laboratory and financed by the United States Air Force. Images were annotated by skilled Civil Air Patrol volunteers following the official FEMA Preliminary Damage Assessment standards [2], making sure the labels are complying with actual FEMA damage evaluation criteria.

The dataset includes 9,969 aerial photographs showing significant disaster categories across 15 states in the US. The initial image resolution is 1800×1200 pixels and the dataset is available to the public on HuggingFace.

B. Label Structure

Each image in the dataset comes with 12 binary label columns describing the infrastructure features visible in the image. Three damage classes were mapped from these columns based on building damage severity, Intact, Major Damage, and Structural Failure which are aligned with the official FEMA Preliminary Damage Assessment scale [2].

C. Class Distribution

The class distribution of the LADI v2 dataset shows the statistical reality of any real-world disasters where most buildings survive. As shown in Table 3, the dataset is highly dominated by the Intact class at 81.5%, while Major Damage accounts for only 5.6% and Structural Failure for 12.9% of the total images. This huge imbalance is not a data collection error but an accurate reflection of how disasters have an impact on the infrastructure in the real world.

TABLE III
OVERALL CLASS DISTRIBUTION ACROSS THE ENTIRE DATASET

Class	Total Images	Percentage	Description
Intact	8,253	82.8%	No visible structural damage
Structural Failure	1,153	11.6%	Complete collapse or destruction
Major Damage	563	5.6%	Significant damage — building still standing
Total	9,969	100 %	

D. Train, Validation, and Test Split

The train, validation, and test split was pre-defined by MIT Lincoln Laboratory following the 80/9/11 ratio. The test set was kept completely untouched throughout the entire project and was never used during training, validation, or hyperparameter tuning.

VI. RESULTS AND ANALYSIS

A. Baseline Performance

The first model trained without any imbalance handling strategies achieved 100% test accuracy by predicting every single image as Intact. Despite appearing impressive at first glance, the Macro F1 score of 0.33 exposes the reality, the model utterly failed to identify any damaged structures. This finding validated that overall accuracy is a deceptive measure for imbalanced datasets and set the standard that all following models had to enhance

TABLE IV
BASELINE MODEL RESULTS — NO IMBALANCE HANDLING

Class	Precision	Recall	F1 Score	Accuracy
Intact	0.95	1.00	0.98	100%
Major Damage	0.00	0.00	0.00	0%
Structural Failure	0.00	0.00	0.00	0%
Macro Average	0.32	0.33	0.33	—

B. Strategy Progression

Seven strategies were applied progressively to address class imbalance. Strategies 1 through 5 individually failed to produce non-zero minority class F1 scores. The first meaningful breakthrough came with Strategy 6 combining Focal Loss with Macro F1 monitoring, which produced the first non-zero Major Damage F1 of 0.11 and Structural Failure F1 of 0.02. Strategy 7 which is targeted augmentation, became the foundation for all subsequent benchmarking models and delivered the strongest overall results.

TABLE V
STRATEGY PROGRESSION — IMPACT ON MINORITY CLASS DETECTION

Strategy	Intact F1	MD F1	SF F1	Macro F1
1 — Baseline	0.98	0.00	0.00	0.33
2 — Class Weights (val_acc)	0.98	0.00	0.00	0.33
3 — Class Weights (val_loss)	0.93	0.00	0.00	0.31
4 — Focal Loss + Balanced Gen	0.98	0.00	0.00	0.33
5 — Threshold Adjustment	0.88	0.00	0.01	0.30
6 — Focal Loss + Macro F1	0.92	0.11	0.02	0.35
7 — Full Combined Pipeline	Best results across all benchmarking models			

C. Model Benchmarking Results

Five CNN models were trained and analysed on the augmented dataset. Table 7 presents the full benchmarking outcomes for all the five models.

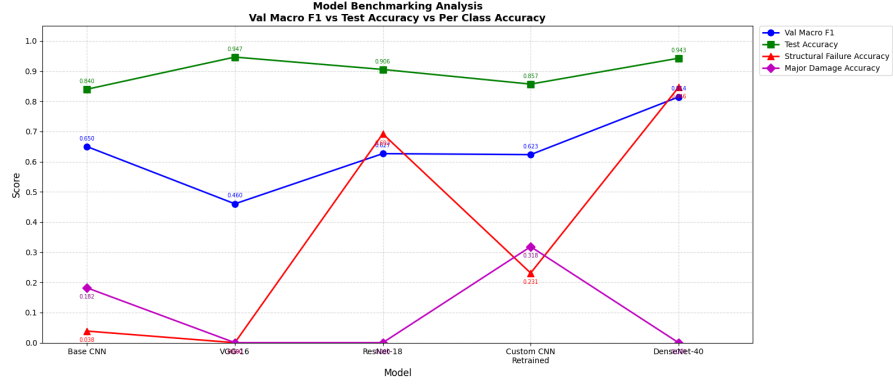


Fig. 1. Benchmark analysis line chart showing Val Macro F1, Test Accuracy, Major Damage Accuracy and Structural Failure Accuracy across all five models.

TABLE VI
COMPLETE MODEL BENCHMARKING RESULTS

Model	Test Acc	Val Macro F1	MD Acc	SF Acc
Custom DamageCNN	85.70%	0.7367	31.82%	23.08%
VGG-16 Inspired	94.66%	0.4602	0.00%	0.00%
ResNet-18 Inspired	90.56%	0.6266	0.00%	69.23%
InceptionV1 Inspired	64.73%	0.59	0.00%	92.31%
DenseNet-40 Inspired	94.28%	0.8143	0.00%	84.62%

D. Individual Model Results

1) *Base Custom CNN*: Custom CNN with 4 convolutional blocks, retrained on the augmented dataset

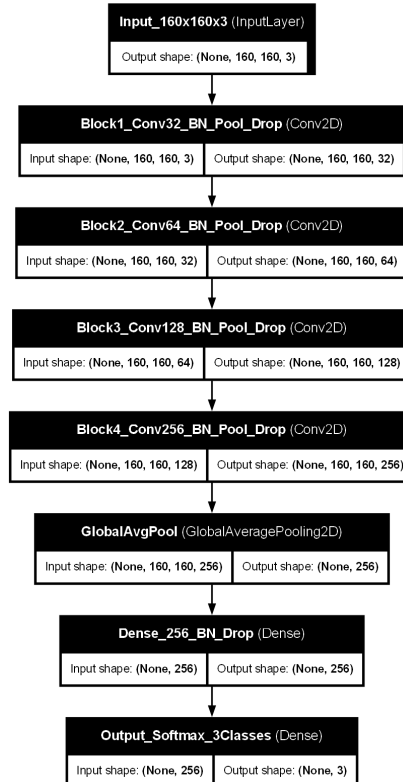


Fig. 2. Retrained Custom DamageCNN architecture — 4 convolutional blocks with filters 32, 64, 128, and 256.

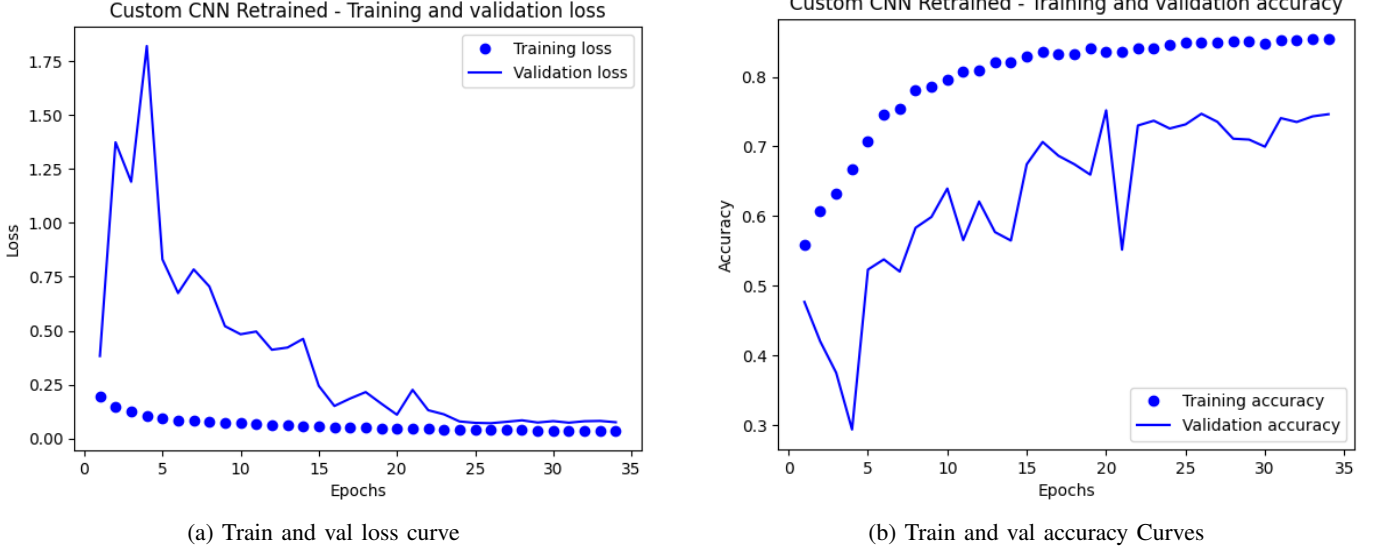


TABLE VII
CLASSIFICATION REPORT — CUSTOM DAMAGECNN RETRAINED

Class	Precision	Recall	F1 Score	Accuracy
Intact	0.97	0.89	0.93	87.81%
Major Damage	0.08	0.45	0.13	31.82%
Structural Failure	0.00	0.00	0.00	23.08%
Macro Average	0.35	0.45	0.35	—

2) *VGG-16 CNN*: 12 Convolutional layers, with triple stacked convolutions in 3 and 4 blocks.

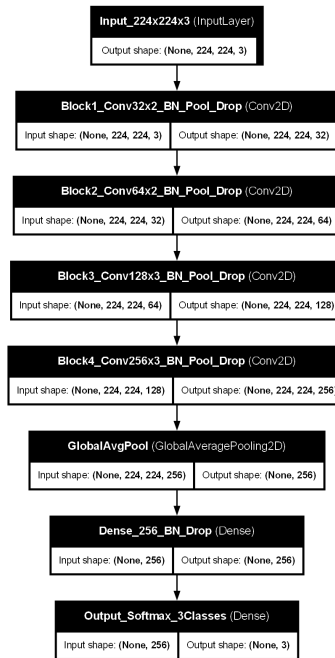
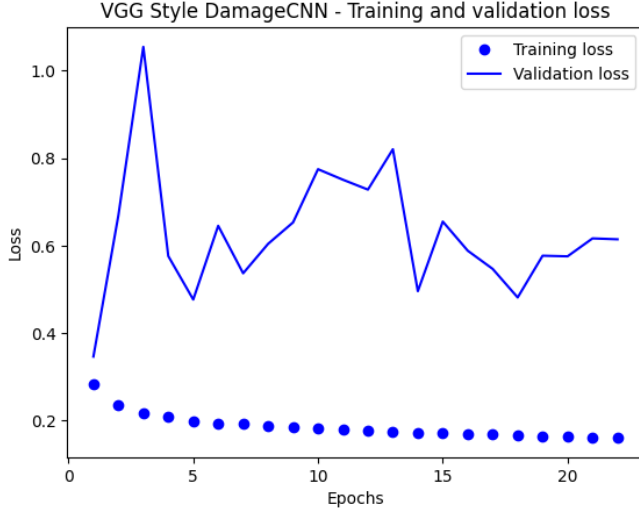
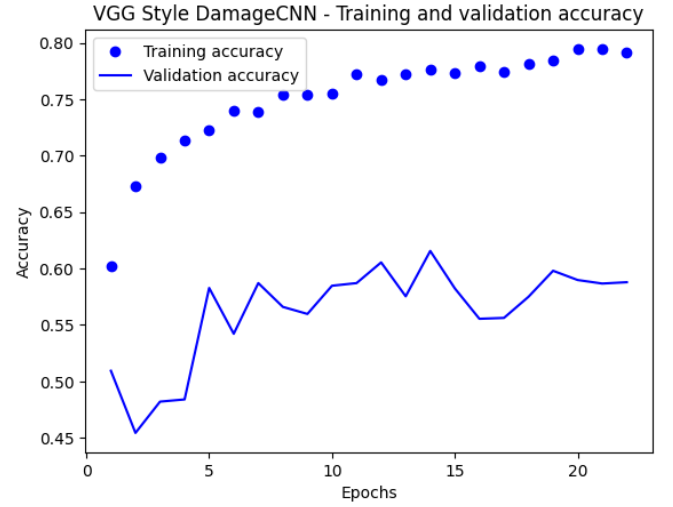


Fig. 3. VGG-16 CNN architecture — 12 convolutional layers with double and triple stacked Conv2D blocks.



(a) Train and val loss curve



(b) Train and val accuracy Curves

TABLE VIII
CLASSIFICATION REPORT — VGG-16 CNN

Class	Precision	Recall	F1 Score	Accuracy
Intact	0.95	0.99	0.97	99.20%
Major Damage	0.00	0.00	0.00	0.00%
Structural Failure	0.00	0.00	0.00	0.00%
Macro Average	0.32	0.33	0.32	—

3) *ResNet-18 CNN*: Residual skip connections across three sets of blocks with 64, 128 and 256 filters.

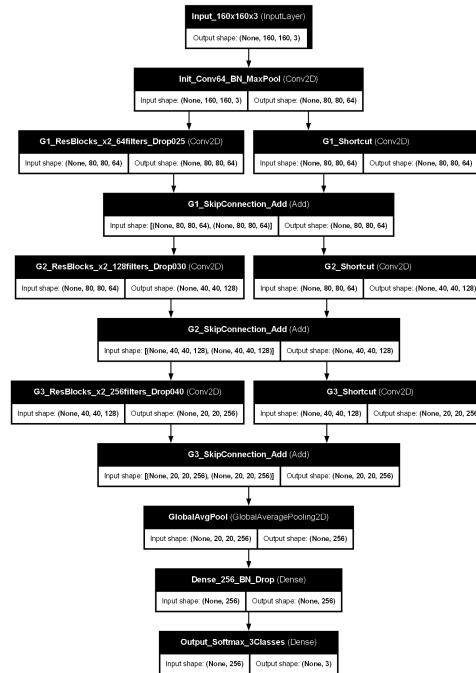


Fig. 4. ResNet-18 CNN architecture — three residual block groups with skip connections.

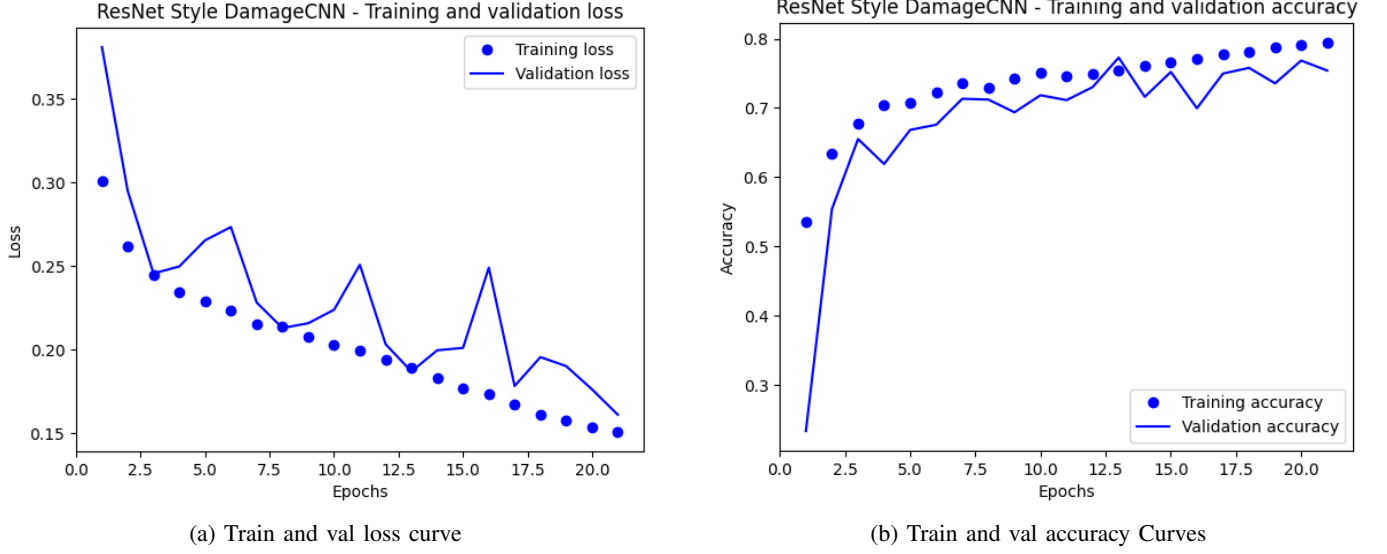


TABLE IX
CLASSIFICATION REPORT — RESNET-18 CNN

Class	Precision	Recall	F1 Score	Accuracy
Intact	0.96	0.95	0.95	60.44%
Major Damage	0.00	0.00	0.00	0.00%
Structural Failure	0.04	0.69	0.08	69.23%
Macro Average	0.34	0.35	0.34	—

4) Inception V1 CNN: Parallel Convolutions at Three different scales

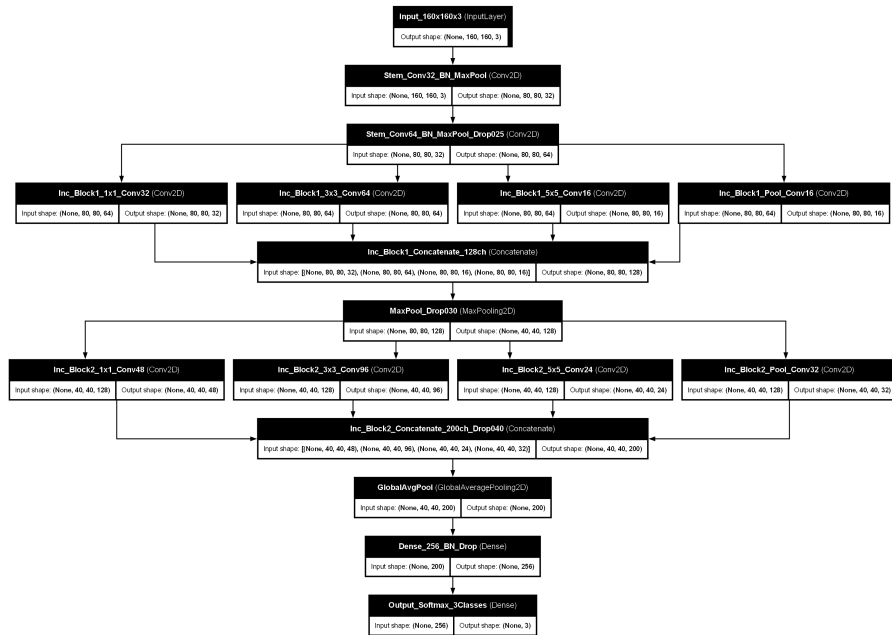
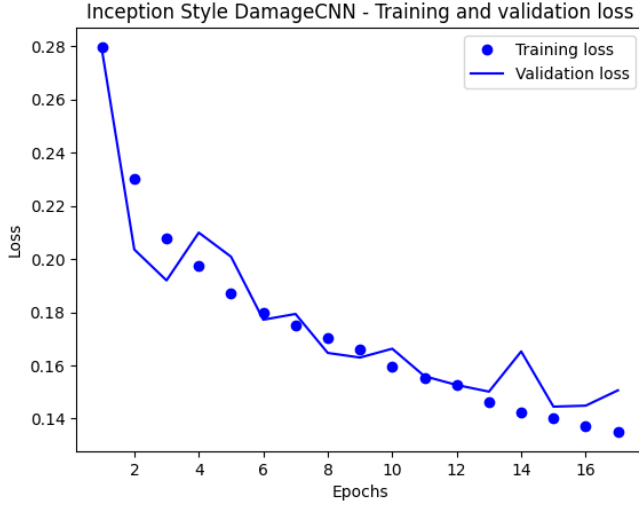
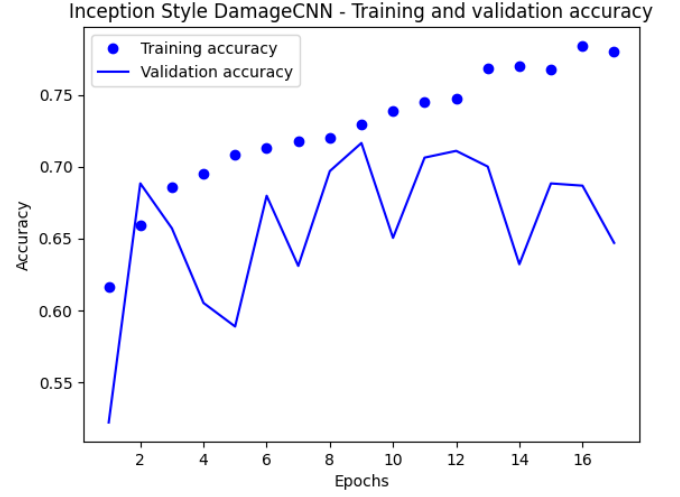


Fig. 5. InceptionV1 CNN architecture — parallel multi-scale convolutions in each inception module.



(a) Train and val loss curve



(b) Train and val accuracy Curves

TABLE X
CLASSIFICATION REPORT — INCEPTIONV1 CNN

Class	Precision	Recall	F1 Score	Accuracy
Intact	0.99	0.66	0.79	36.16%
Major Damage	0.00	0.00	0.00	0.00%
Structural Failure	0.05	0.69	0.09	92.31%
Macro Average	0.34	0.45	0.29	—

5) DenseNet-40 CNN: Four Dense Blocks with 4, 6, 6, 4 layers

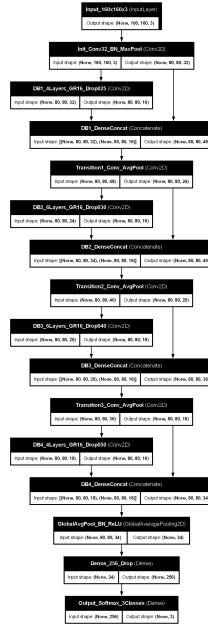
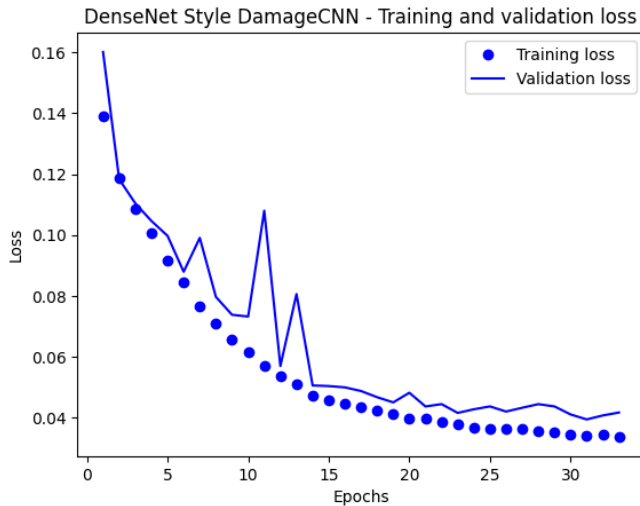
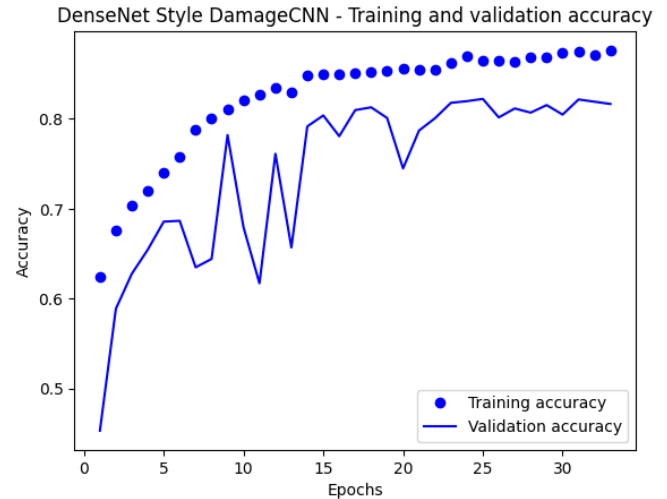


Fig. 6. DenseNet-40 CNN architecture — four dense blocks with dense connections between all layers.



(a) Train and val loss curve



(b) Train and val accuracy Curves

TABLE XI
CLASSIFICATION REPORT — DENSENET-40 CNN

Class	Precision	Recall	F1 Score	Accuracy
Intact	0.95	0.99	0.97	99.20%
Major Damage	0.17	0.05	0.07	0.00%
Structural Failure	0.00	0.00	0.00	84.62%
Macro Average	0.37	0.34	0.35	—

E. GRAD-CAM Visualization Results

GRAD-CAM visualizations were generated for the Custom DamageCNN Retrained model across all three damage classes. The heatmaps show which regions of each aerial image the model focused on when making its prediction.

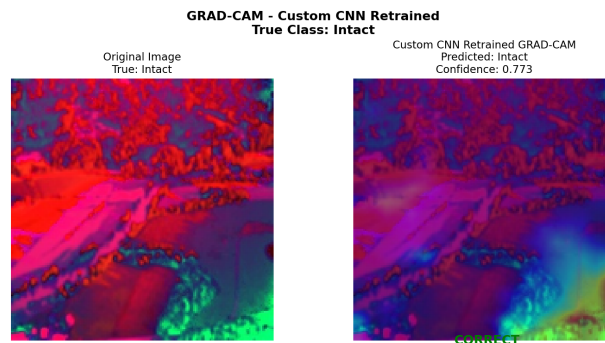


Fig. 7. GRAD-CAM visualization for a correctly predicted Intact image. The model focuses on undamaged roof and wall structures.

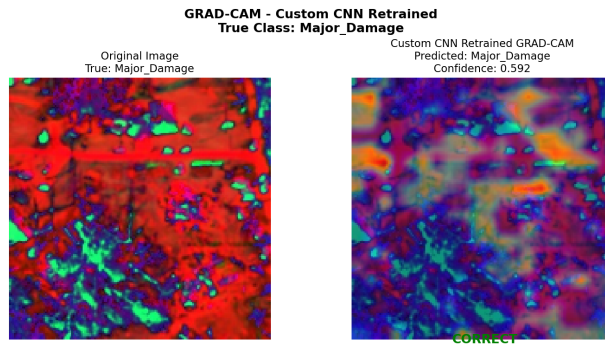


Fig. 8. GRAD-CAM visualization for a correctly predicted Major Damage image. The model focuses on damaged structural sections.

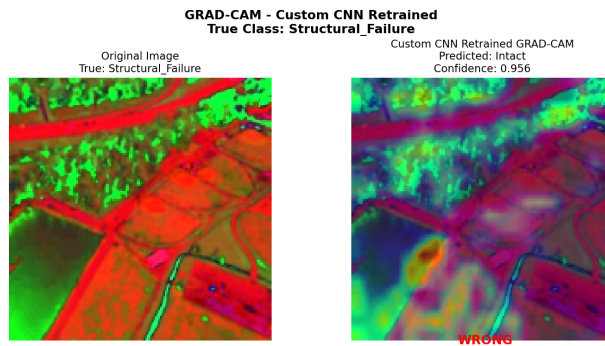


Fig. 9. GRAD-CAM visualization for a Structural Failure image. Even for incorrect predictions the model focuses on structurally relevant regions.

F. Discussion

The benchmarking results reveal several important findings. First, architectural depth alone is not sufficient for minority class detection, the VGG-16 Inspired model had 12 convolutional layers but achieved zero minority class detection when trained on unaugmented data. Second, targeted augmentation was as important as architectural choice, the Custom DamageCNN Retrained on augmented data achieved 31.82% Major Damage accuracy using the same simple architecture that previously detected nothing. Third, DenseNet's dense connectivity proved most effective for overall performance, its feature reuse mechanism produced a 133% improvement in validation Macro F1 from the baseline of 0.33 to 0.8143. Fourth, the small minority class test set 22 and 26 images, limits the test-time accuracy estimates significantly, making validation Macro F1 the more reliable performance indicator throughout this project.

VII. FUTURE WORK

This project showed promising results but there are several areas that can be improved in the future.

The most important next step is using transfer learning like, starting with models that have already been trained on large image datasets rather than building everything from scratch. This would give the model a much better starting point and would likely improve minority class detection significantly.

Another important direction is, collecting more labeled disaster images, especially for Major Damage and Structural Failure. With only 22 and 26 test images for these classes, the test results are not much reliable. More amount of data would give a much clearer picture of how well the models actually perform.

Exploring Vision Transformer architectures is also worth considering, as they have shown strong results on aerial imagery classification in recent research and could outperform the CNN-based approach used here.

Finally, the system could be deployed as a real-time tool integrated with FEMA's existing infrastructure, where aerial images captured during disaster missions are automatically classified and mapped to help emergency teams identify the most damaged areas immediately.

VIII. CONCLUSION

This project built an automated system to classify aerial disaster images into three damage categories such as, Intact, Major Damage, and Structural Failure. using five custom CNN architectures trained completely from scratch on the LADI v2 federal government dataset [1].

Starting from a baseline Macro F1 of 0.33 where the model predicted almost everything as Intact, seven strategies were applied together to address severe class imbalance. The result was a 133% improvement in validation Macro F1, reaching 0.8143 with the DenseNet-40 CNN [6] correctly identifying 84.62% of completely destroyed buildings. The Custom DamageCNN Retrained on augmented data achieved the best Major Damage detection at 31.82%, showing that targeted data augmentation was just as important as architectural design.

The key finding of this project is that no single factor was responsible for the improvement; it took the right architecture, the right loss function [7], the right augmentation strategy [9], and the right evaluation metric working together to produce meaningful results on a severely imbalanced real-world dataset.

The complete pipeline with preprocessing, model training, and GRAD-CAM explainability [11] provides a strong foundation for a real-time automated damage assessment tool that could help FEMA emergency teams identify the most critically damaged areas faster after a disaster.

REFERENCES

- [1] S. Scheele, K. Picchione, and J. Liu, "Ladi v2: Multi-label dataset and classifiers for low-altitude disaster imagery," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 2235–2243.
- [2] Fema, "Fema preliminary damage assessment guide," 2020.
- [3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [5] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.
- [6] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.
- [7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2980–2988.
- [8] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural networks*, vol. 106, pp. 249–259, 2018.
- [9] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of big data*, vol. 6, no. 1, pp. 1–48, 2019.
- [10] J. L. Leevy, T. M. Khoshgoftaar, R. A. Bauder, and N. Seliya, "A survey on addressing high-class imbalance in big data," *Journal of Big Data*, vol. 5, no. 1, p. 42, 2018.
- [11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: visual explanations from deep networks via gradient-based localization," *International journal of computer vision*, vol. 128, no. 2, pp. 336–359, 2020.
- [12] M. O. Yusuf, M. Hanzla, N. Al Mudawi, T. Sadiq, B. Alabdullah, H. Rahman, and A. Algarni, "Target detection and classification via efficientdet and cnn over unmanned aerial vehicles," *Frontiers in Neurorobotics*, vol. 18, p. 1448538, 2024.
- [13] T. Jintasuttisak, P. Chabplan, S. Issaro, O. Saeung, and T. Suwanroj, "Accurate segmentation of vegetation in uav desert imagery using hsv-glm features and svm classification," *Journal of Imaging*, vol. 12, no. 1, p. 9, 2025.
- [14] A. M. Abdelmonem, M. M. Ata, A. A. Atey, A. Shaalan, and R. A. El-Sayed, "High-performance scene classification in remote sensing imagery using a custom deep cnn architecture," *Scientific Reports*, vol. 16, no. 1, p. 5946, 2026.